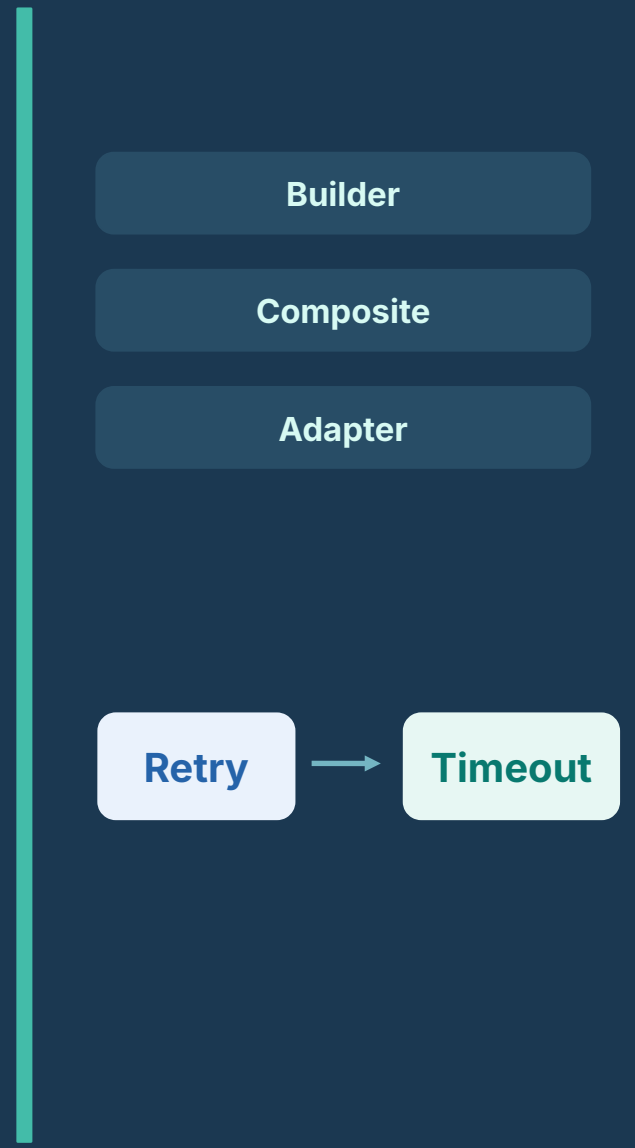
РЕВЕРС-ИНЖИНИРИНГ · .NET

Как устроен Resilience Pipeline в Polly 8.9.0

Построение, выполнение и архитектурные решения на примере Retry → Timeout



Исходники: tag 8.9.0 · 1fc5d11

Что такое Polly и зачем она нужна

Библиотека устойчивости для .NET: управляет выполнением операций при временных сбоях.

Проблема

Внешний HTTP-сервис отвечает нестабильно; сетевые вызовы могут завершаться ошибкой или зависать.

Решение

Политики устойчивости задают поведение при сбое: повторить, ограничить время, прекратить попытки.

Применение

HTTP и RPC, базы данных, очереди сообщений и другие внешние зависимости.

Почему изучаем именно Pipeline?

Несколько самостоятельных стратегий образуют одну управляемую композицию — и порядок меняет поведение.

Что исследуем и какой пример используем

Конкретный сценарий: два сбоя, затем успех; внутри каждой попытки находится Timeout.

```
var pipeline = new ResiliencePipelineBuilder()
    .AddRetry(new RetryStrategyOptions {
        MaxRetryAttempts = 2,
        Delay = TimeSpan.FromSeconds(1)
    })
    .AddTimeout(TimeSpan.FromSeconds(5))
    .Build();

await pipeline.ExecuteAsync(operation);
```

Внутри реализации

01 Регистрация
Сохранить фабрики и options

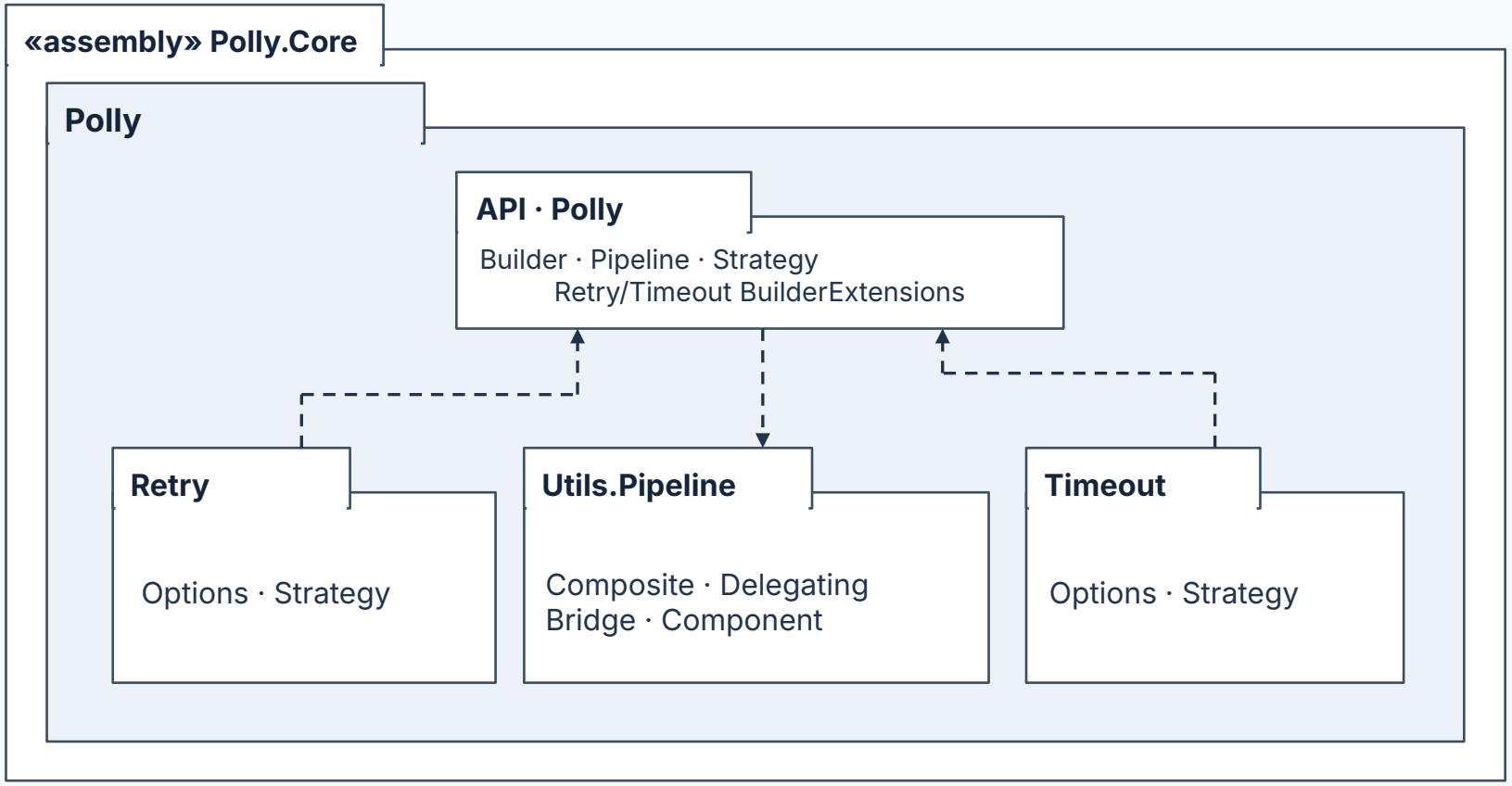
02 Построение
Создать и связать компоненты

03 Выполнение
Передавать callback и Outcome

Build один раз · ExecuteAsync — многократные попытки

Архитектура Polly.Core: ключевые пространства имён

Вложенные пакеты показывают пространства имён, пунктирные стрелки — зависимости.

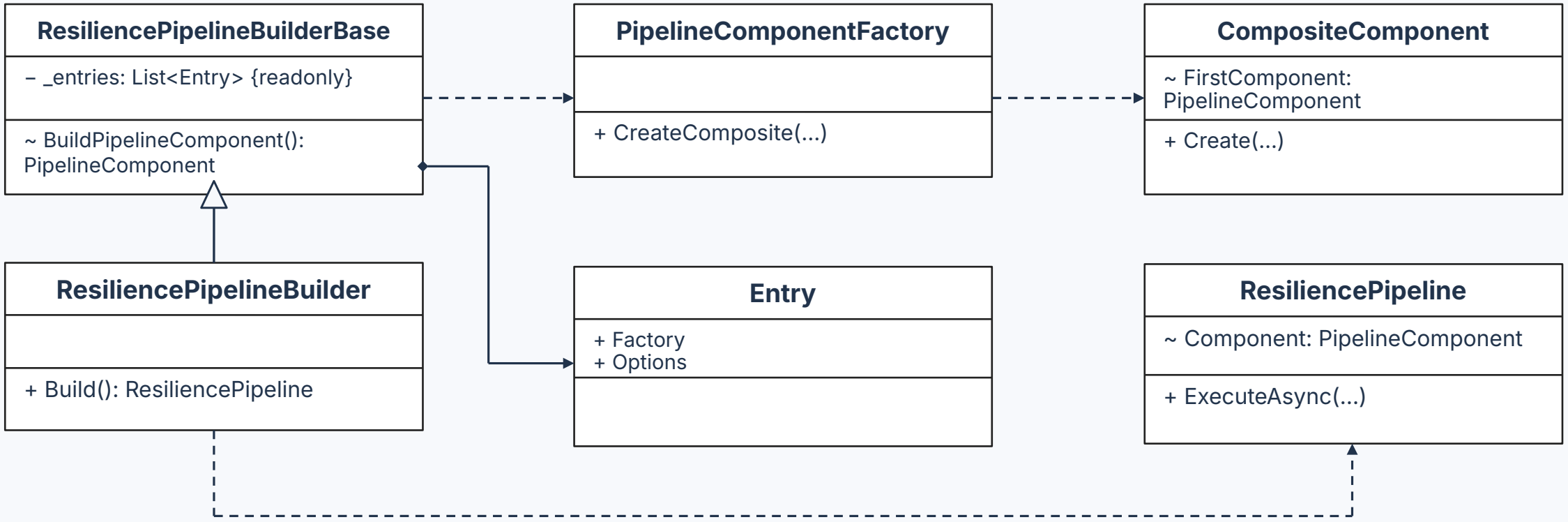


Зависимости

- Retry и Timeout используют общие контракты Polly.
- Builder создаёт внутренние PipelineComponent.
- Utils.Pipeline исполняет связанную цепочку.

Построение Pipeline: классы и ответственности

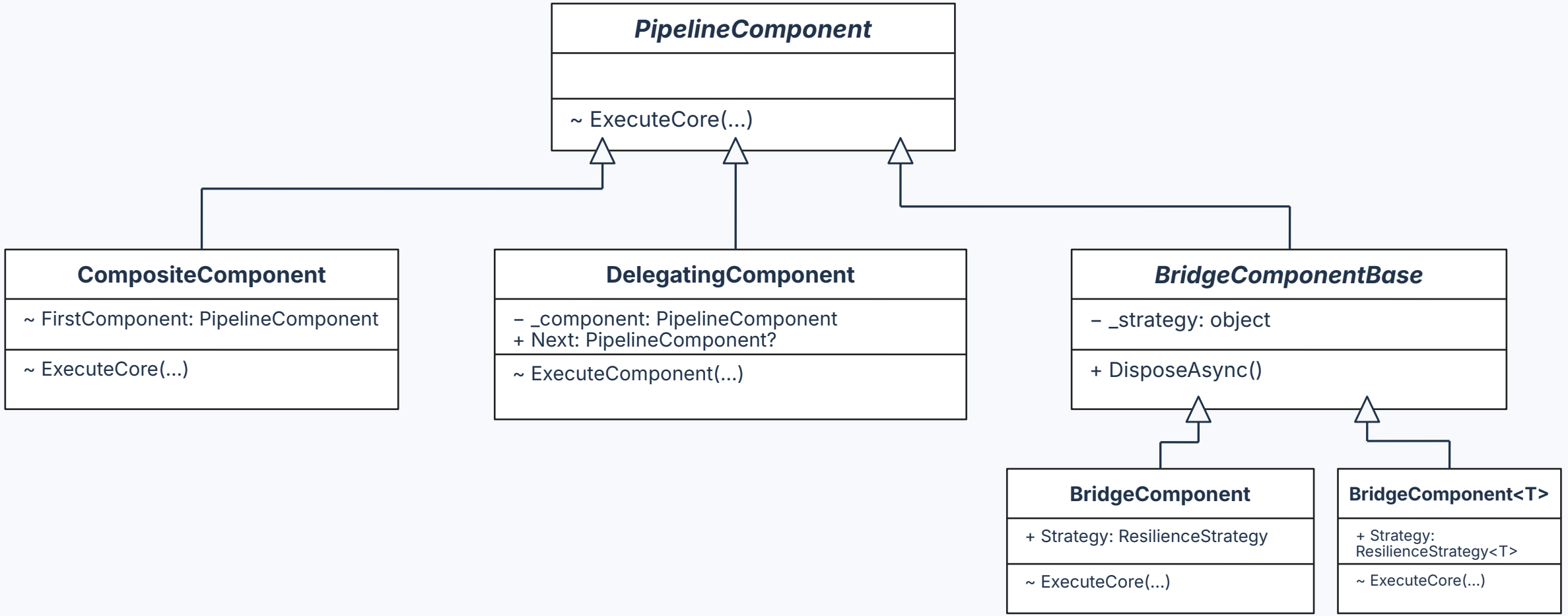
Builder описывает Pipeline, фабрики превращают конфигурацию в исполняемые компоненты.



При Build создаётся ResiliencePipeline с корневым CompositeComponent.

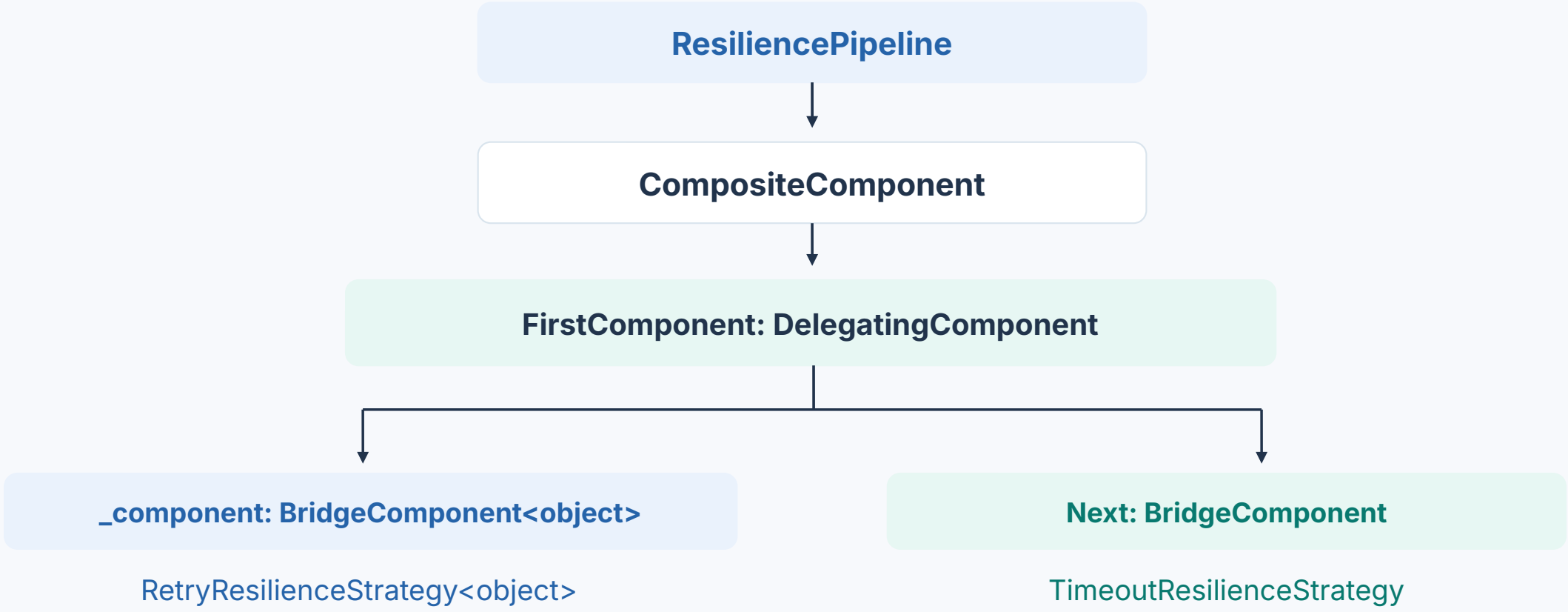
Выполнение: иерархия и адаптеры

Общий компонент задаёт исполнение; Bridge соединяет его с реализациями стратегий.



Что реально лежит внутри Pipeline после Build()

Снимок структуры объектов для AddRetry → AddTimeout; это не повтор диаграммы наследования.



Retry видит continuation, а не конкретный класс Timeout.

Какие паттерны подтверждаются исходниками

Паттерны, подтверждённые реализацией компонентов Polly.

ПАТТЕРН	КЛАСС / МЕХАНИЗМ	ЗАЧЕМ НУЖЕН
Builder	**ResiliencePipelineBuilder**	Регистрирует шаги и создаёт продукт при Build()
Composite	**CompositeComponent**	Единая точка выполнения составной цепочки
Strategy	**Retry / Timeout**	Разные алгоритмы управления выполнением
Adapter	**BridgeComponent<T>**	Приводит контракты стратегий к PipelineComponent

Расширение Новые алгоритмы подключаются через ResilienceStrategy и AddStrategy, без изменения ядра Pipeline.

Почему композиция построена через callback?

Одно архитектурное решение, подробно раскрытое по пяти требованиям задания.

1	**1. Проблема**	Retry должен повторять всю внутреннюю цепочку; Timeout ограничивать нужный участок.
2	**2. Устройство**	Composite + Delegating + два Bridge; continuation передаёт управление дальше.
3	**3. Преимущество**	Стратегии не знают соседей; порядок регистрации меняет область действия Timeout.
4	**4. Ограничения**	Обёртки, сложная трассировка вызовов, адаптация generic-результатов.
5	**5. Альтернатива**	Массив компонентов и индексный ExecuteAt(i), сохраняющий continuation.

Идея: стратегия получает «то, что осталось выполнить», как callback, а не ссылку на Timeout по имени.

Почему не обычный foreach? И что можно иначе?

Три способа организовать передачу управления между стратегиями.

Обычный foreach

- Простая линейная модель
- Retry не может повторить вложенный Timeout
- Неэквивалентен нашему сценарию

Недостаточно

Текущий Polly

- DelegatingComponent + Next
- Вложенный continuation
- Отдельные Bridge и обходы вызовов

Связанная композиция

ExecuteAt(i)

- Массив вместо связанного Next
- ExecuteAt(i+1) — продолжение
- Те же задачи типов и контекста

Реальная альтернатива

При сохранении контрактов Polly индексированный исполнитель тоже требует продолжений и адаптации типов.

Как Composite связывает компоненты

CompositeComponent.Create() связывает адаптеры стратегий в цепочку.

```
delegatingComponents[^1].Next = components[^1];


for (var i = 0; i < delegatingComponents.Length - 1; i++)

    delegatingComponents[i].Next = delegatingComponents[i + 1];


return new CompositeComponent(

    delegatingComponents[0], components, telemetry, timeProvider);
```

Источник: Polly/src/Polly.Core/Utils/Pipeline/CompositeComponent.cs, Create(), строки 40–58.

Роль метода

- Для двух стратегий создаётся один DelegatingComponent.
- Next первого звена указывает на Timeout Bridge.
- Composite хранит корень исполнения и список компонентов.

Как callback доходит до Timeout

Точный путь: ExecuteNext распознаёт негeneric Bridge и вызывает Strategy.ExecuteCore напрямую.

```
// DelegatingComponent.ExecuteNext(...)

if (next is BridgeComponent bridge)

    return bridge.Strategy.ExecuteCore(

        callback, context, state);
```

Что происходит при повторе

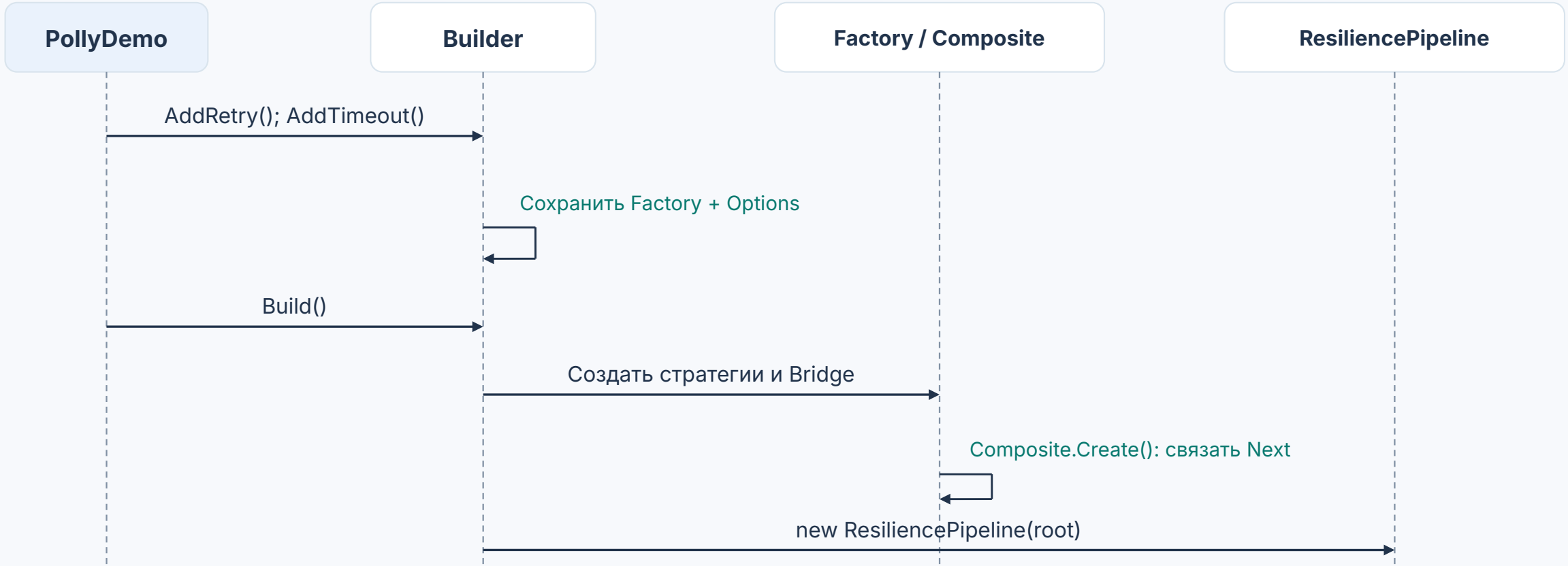
1. Retry вызывает continuation.
2. ExecuteNext передаёт управление Timeout.
3. Outcome возвращается для решения о повторе.

Источник: Polly/src/Polly.Core/Utils/Pipeline/DelegatingComponent.cs, ExecuteNext(), строки 55–79.

ExecuteNext напрямую запускает стратегию Timeout; Retry остаётся независимым от её типа.

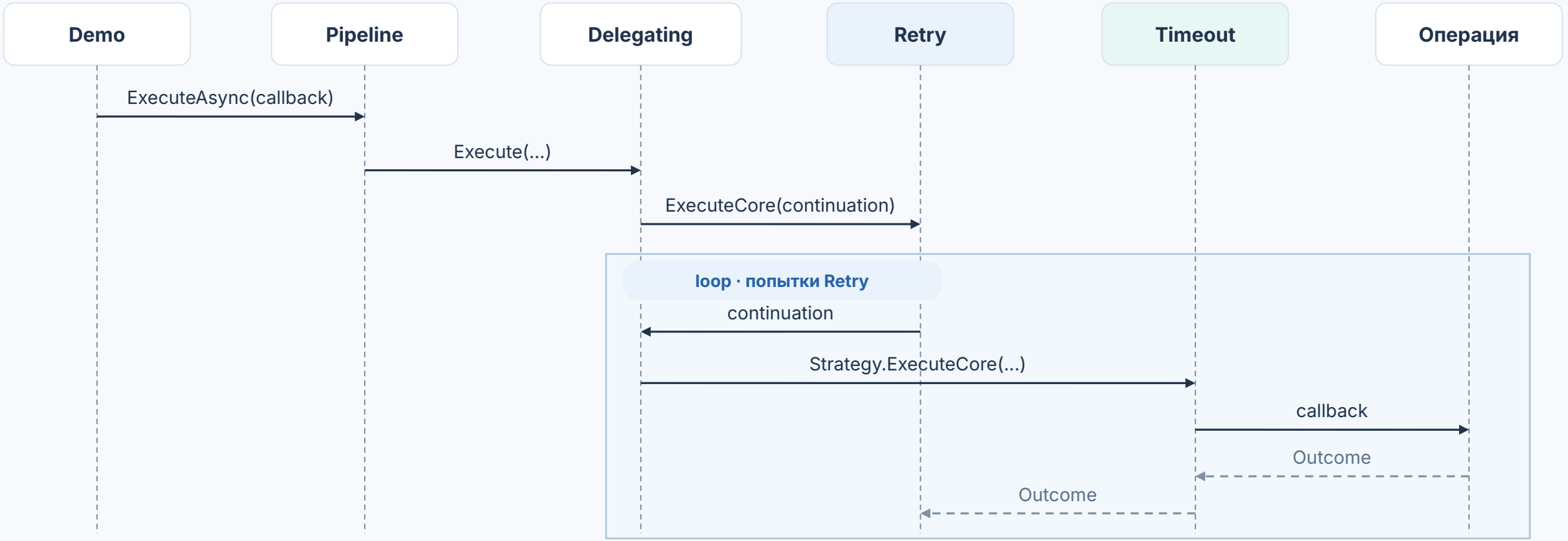
Сценарий I. Регистрация и Build()

Фабрики сохраняются при AddRetry/AddTimeout; объекты создаются при Build().



Сценарий II. ExecuteAsync() и повтор

Retry вызывает continuation повторно, не перестраивая Pipeline.



Retry проверяет ShouldHandle и лимит попыток.